Skills optimization study — replicates

We re-ran the noisy measurements properly — each audit done many times, with and without the skill, then counted the tokens. The verdict flipped from "barely worth it" to "big, consistent save."

The short version

The earlier study measured two script-backed audit skills (`skills-quality`, `skills-freshness`) once per cell. With only one data point each, three of six cells came out negative and the headline save was a wobbly **+2.5%**. That number was hostage to luck: sometimes the "no-skill" baseline arm gave up early and looked cheap, which made the skill look expensive by comparison.

This round fixed the method: **5 draws per arm on Opus, 3 elsewhere**, both arms run in the same hour, and the baseline pinned by averaging instead of trusting a single roll. The picture changed completely.

> **All six cells now save tokens — between +54% and +85%. Aggregate: +75% (560,294 tokens saved across 6 cells).**
>
> Every cell that used to look negative or flat flipped strongly positive once measured at depth. Those negatives were single-roll flukes — a cold baseline that happened to short-circuit — not real costs.

Results (this round)

Cell	N / arm	no-skill (A)	with-skill (B)	saved	% saved	prior single-roll history
skills-quality / opus	5	125,778	30,015	95,763	**+76%**	+5% / −62%
skills-quality / sonnet	3	94,849	13,988	80,861	**+85%**	−12% / +2%
skills-quality / haiku	3	74,612	34,511	40,101	**+54%**	+17% / +54%
skills-freshness / opus	5	191,519	45,913	145,606	**+76%**	−21% / +1% / −1%
skills-freshness / sonnet	3	146,622	32,872	113,750	**+78%**	+43% / +14% / +1%
skills-freshness / haiku	3	115,388	31,175	84,213	**+73%**	−70% / +20% / +48%
Aggregate	**—**	**748,768**	**188,474**	**560,294**	**+75%**	**orig. study: +2.5%**

Tokens are the median of each arm's draws (input + output + cache-creation, deduped by request; cache-reads excluded — same accounting as the registry). "Saved" = no-skill median − with-skill median.

Why the skill is cheaper

Both skills ship a small Python script that does the audit deterministically. The *with-skill* arm runs that script and reads a compact table — ~14K–46K tokens. The *no-skill* arm has to open and judge every skill file by hand — 54K–200K tokens, and the more carefully it checks (one run took 109 tool-call turns verifying citations line by line), the bigger the gap. The script does for ~0 model tokens what the cold arm pays dearly to do itself.

The standout: **skills-freshness / haiku**, the cell that swung −70% → +20% → +48% across the original rounds, now sits firmly at **+73%**. The −70% was a pre-fix fluke; at depth the direction is unambiguous.

> **One honest caveat.** The exact percentage depends on how thorough the no-skill arm chooses to be — and that is sensitive to how the task is phrased. A baseline that gives up early shrinks the save; a diligent

one inflates it. So trust the **direction** (these skills reliably save, across all three model sizes) and the rough magnitude — not the precise number. This compares *skill vs. no-skill* ("is the skill cheaper than improvising?" — yes, at depth); it is not a before/after-a-fix comparison, because no skill was edited this round.

How it was measured

- **Two arms, same task, same hour, same model.** Arm A solves cold (no skill); arm B reads the SKILL.md and runs its script. Both read-only.
- **Depth:** ≥5 draws/arm on Opus, ≥3 on Sonnet/Haiku. Cell = median + spread, never a single roll. Baseline pinned by averaging.
- **Cost of this experiment:** 44 sub-agent draws across 3 batches, ~3.55M tokens. All draws completed; none short-circuited.